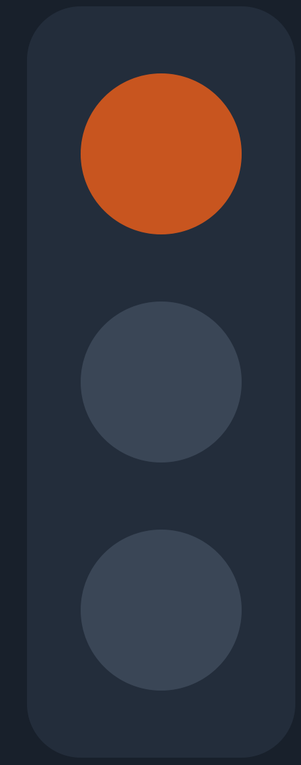
PROGRAMACIÓN CONCURRENTE

Cómo usar jBACI

Qué hace cada cuadro del simulador, y un ejemplo completo: cuatro autos que comparten un cruce protegido por un semáforo.



QUÉ ES

Un simulador para ver procesos concurrentes paso a paso

</>

Editor

Escribes el programa en C-- (archivo **.cm**) o en Pascal (**.pm**).

Compilador BACI

Traduce el código a **PCode**, instrucciones de una máquina virtual simple.

Intérprete

Ejecuta el PCode y muestra cada proceso, sus variables y quién está bloqueado.

BACI = *Ben-Ari Concurrent Interpreter*. jBACI es la versión en Java con interfaz gráfica, creada por Moti Ben-Ari para enseñar.

LOS DOS MODOS

jBACI siempre está en modo Edit o en modo Run

1 · EDIT

Escribir

Abrir, editar y guardar el .cm

→

2 · COMPILE

Compilar

Revisa errores y genera el PCode

→

3 · RUN

Ejecutar

Go, Pause o paso a paso

→

4 · EDIT

Volver

Corregir y repetir el ciclo

En cada modo solo se activan los botones que sirven ahí. Los botones **Edit** y **Run** cambian de un modo al otro.

MODO EDIT

Los botones del modo edición

Botón	Qué hace
Open / New	Abre un archivo .cm existente o crea uno vacío
Save / Save as	Guarda el archivo (o con otro nombre)
Copy · Cut · Paste	Copiar, cortar y pegar texto, como en cualquier editor
Find / Find again	Busca un texto en el código y la siguiente coincidencia
Compile	Compila con bacc.exe (.cm) o bapas.exe (.pm) según la extensión
Run	Pasa al modo ejecución con el programa ya compilado

COMPILE

Compilar crea dos archivos nuevos

El resultado sale en una ventana emergente; se cierra con **Enter**. Si hay errores, el cursor salta a la línea del primero.

`.pco`

El PCode: el programa traducido que ejecuta el intérprete.

`.lst`

El listado: cada línea del código con su dirección de PCode (pc).

02-trafico-cruce-uno-a-la-vez.lst

```
line  pc
  10   0  binarysem cruce = 1;
  16   5    wait(cruce);
  18   7    cout << "...entra";
  24  39    signal(cruce);
  28  43    cobegin {
  29  44      autoQueCruza(1);
  30  48      autoQueCruza(2);
```

MODO RUN

El mapa de la ventana de ejecución

1 Barra de botones: Go · Pause · Step Source · Step PCode · Edit

2

Tabla de procesos

Una fila por proceso: activo, terminado o bloqueado

3

Ventanas de proceso

Una por proceso: código con flecha verde, PCode y variables locales

4

Ventanas del sistema

Console · Globals · History · Linda

CUADRO 1 · BARRA DE BOTONES

Controlas la ejecución a tu ritmo

Botón	Qué hace
Go	Ejecuta el programa de corrido
Pause	Detiene la ejecución donde va
Run	Reinicia el programa desde el principio
Step Source	Avanza una línea del código fuente
Step PCode	Avanza una sola instrucción de PCode
Next / Previous	Elige qué proceso ejecuta el próximo paso
Edit	Vuelve al modo edición

CUADRO 2 · TABLA DE PROCESOS

Una fila por proceso, con su estado

Número y nombre del proceso.

Active / Finished: si está corriendo o ya terminó.

Suspended: se llena cuando el proceso está bloqueado en un semáforo o un monitor.

Monitor: el monitor en que entró, si usa monitores.

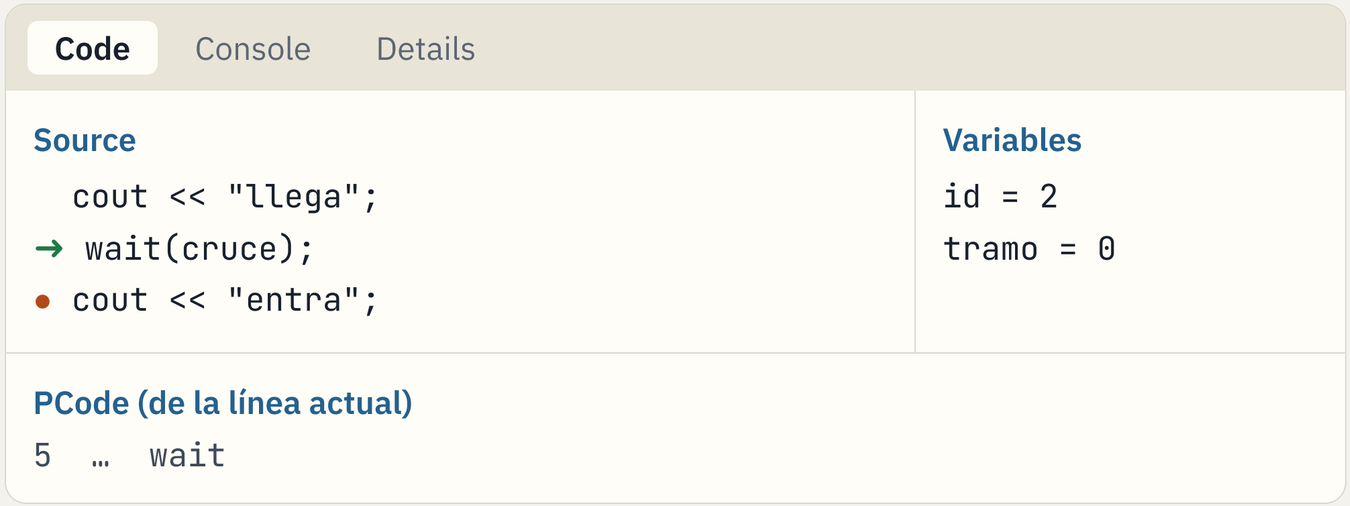
Fila resaltada: el proceso que ejecuta la siguiente instrucción.

#	Proceso	Estado
1	autoQueCruza	activo, en el cruce
2	autoQueCruza	suspendido (cruce)
3	autoQueCruza	suspendido (cruce)
4	autoQueCruza	suspendido (cruce)

Ejemplo ilustrativo con el programa 02: un auto cruza y los otros tres esperan en el semáforo.

CUADRO 3 · VENTANA DE PROCESO

Cada proceso tiene su propia ventana



Flecha verde: la próxima línea que ejecutará este proceso.

Punto rojo: un breakpoint.

Console: solo lo que imprimió este proceso.

Details: la pila del proceso.

CUADRO 4 · VENTANAS DEL SISTEMA

Cuatro ventanas que ven el programa completo

Console

La salida de todos los procesos juntos: lo que imprimen los `cout`.

Globals

Las variables globales. Aquí se ve el valor del semáforo **cruce**.

History

Las últimas 150 instrucciones ejecutadas, de qué proceso venía cada una.

Linda

El espacio de tuplas, para programas que usan primitivas Linda.

BREAKPOINTS

Un breakpoint frena el programa en una línea

1

Clic en la línea

En el Source o el PCode de una ventana de proceso.

2

Botón Add

Arriba del código. Aparece un punto rojo junto a la línea.

3

Go

Al llegar, se detiene y la flecha verde se pone roja.

4

Remove

Clic en la línea y Remove para quitarlo.

En el ejemplo: un breakpoint en `signal(cruce)` muestra el momento exacto en que se libera el cruce.

MENÚS OPTIONS Y WINDOW

Opciones que cambian cómo se ve la ejecución

Opción	Qué hace
Pause on Process Swap	Pausa cada vez que la CPU cambia de un proceso a otro
Show Active Window	Trae al frente la ventana del proceso que se ejecuta
History on Source Step	History guarda líneas de código en vez de PCode
Write History File	Guarda toda la historia en un archivo .his
Window (menú)	Muestra u oculta Console, Globals, History y Linda

CONEXIÓN CON LA MATERIA

Lo que vimos en clases, visible en jBACI

Concepto (apuntes)	Dónde se ve en jBACI
Estados del proceso (Cap 3)	Tabla de procesos: activo, suspendido (en espera) o terminado
Cola de espera (Cap 3)	Procesos detenidos en un `wait`, marcados en Suspended
PCB y contador de programa (Cap 3)	Flecha verde, variables locales y pila (pestaña Details)
Cambio de contexto (Cap 3)	Options › Pause on Process Swap lo detiene en cada cambio
Planificador preemptive (Cap 5)	El intérprete elige qué proceso sigue; el orden cambia en cada ejecución

Fuente: apuntes Cap 3 (Procesos) y Cap 5 (Planificación de CPU), Silberschatz 10ª ed.

EJEMPLO · 02-TRAFICO-CRUCE-UNO-A-LA-VEZ.CM

Cuatro autos, un cruce, un semáforo

```
binarysem cruce = 1;

void autoQueCruza(int id) {
  cout << "Auto " << id << " llega";
  wait(cruce);
  cout << "... entra al cruce";
  for (tramo = 1; tramo <= 3; tramo++)
    cout << "... cruzando";
  cout << "... sale del cruce";
  signal(cruce);
}

void main() {
  cobegin { autoQueCruza(1); ... autoQueCruza(4); }
}
```

binarysem cruce = 1

1 = cruce libre, 0 = ocupado

wait(cruce)

Si vale 0, el auto se bloquea

signal(cruce)

Libera el cruce y despierta a otro

cobegin

Lanza los 4 autos en paralelo

EJEMPLO · LA DEMO

Seis pasos para correr el ejemplo

PASO 1

Abrir

jbaci.cmd → Open → ejercicios\02-trafico-cruce-uno-a-la-vez.cm

PASO 2

Compile

Sin errores; Enter cierra la ventana del compilador

PASO 3

Run

Aparecen la tabla de procesos, Console y Globals

PASO 4

Step Source

Avanzar línea a línea hasta que los autos llegan al cruce

PASO 5

Observar

cruce en Globals y la columna Suspended

PASO 6

Go

Terminar de corrido; Edit para volver al código

EJEMPLO · QUÉ MIRAR

Tres cuadros cuentan la historia del semáforo

GLOBALS

1 → 0

cruce baja a 0 cuando un auto entra y vuelve a 1 cuando sale y nadie espera.

TABLA DE PROCESOS

3 en espera

Mientras un auto cruza, los otros quedan suspendidos en el semáforo.

CONSOLE

1 a la vez

Entre "entra" y "sale" de un auto solo hay tramos de ese mismo auto.

EXPERIMENTO FINAL

Sin wait y signal, los autos chocan

Comenta las dos líneas, compila y ejecuta. En Console aparecen tramos de dos autos mezclados: dos procesos dentro de la sección crítica al mismo tiempo.

```
// wait(cruce); ... // signal(cruce);
```

OTRA SOLUCIÓN · TRAFFICLIGHT.CM

Dos semáforos que se turnan con una variable

```
int turno = 1;

void semaforoA() {
  while (1) {
    if (turno == 1) {
      cout << "[A]: VERDE";
      cout << "[A]: AMARILLO";
      cout << "[A]: ROJO";
      turno = 2;
    }
  }
}
// semaforoB igual, con turno == 2
```

int turno

Variable compartida: dice a quién le toca

while (1) + if

Pregunta una y otra vez si ya es su turno

turno = 2

Al terminar, le cede el turno al otro

Funciona: A y B nunca están en verde a la vez. Pero usa **espera activa**.

COMPARACIÓN EN JBACI

Espera activa contra semáforo: la tabla de procesos lo muestra

	trafficlight.cm (variable turno)	04 (semáforos verdeNS / verdeEO)
El que espera…	sigue corriendo y preguntando	se bloquea en wait
Tabla de procesos	ambos siempre activos	el que espera aparece suspendido
Uso de CPU	se gasta en vueltas inútiles	solo trabaja quien tiene el turno
History	se llena del mismo if repetido	solo instrucciones útiles
¿Termina?	no: while (1), se detiene con Pause	sí, cuando cruzan los 6 autos

PROBLEMA DE EXAMEN · EXAMEN SO 2025-1, PROBLEMA 4

Estacionamiento de 5 cupos con una sola barrera

```
semaphore cupos = 5;
binarysem barrera = 1;

void autoResidente(int id) {
  wait(cupos);        // espera un espacio
  wait(barrera);  dentro++;  signal(barrera);
  ...                 // estacionado
  wait(barrera);  dentro--;  signal(barrera);
  signal(cupos);      // libera el espacio
}
main: cobegin { 7 autos }
```

cupos = 5

Semáforo **general**: cuenta espacios libres

barrera = 1

Semáforo **binario**: un auto por la barrera

Con 7 autos, los autos 6 y 7 esperan en `wait(cupos)` hasta que alguien salga.

Archivo: ejercicios\05-estacionamiento-examen-2025-1.cm (versión C-- de la pauta en Python y C)

DE LA PAUTA A JBACI

Las mismas operaciones con otro nombre

Operación	Python (pauta)	C con pthreads (pauta)	C-- en jBACI
Crear	Semaphore(5)	sem_init(&c, 0, 5)	semaphore c = 5
Pedir	acquire()	sem_wait(&c)	wait(c)
Liberar	release()	sem_post(&c)	signal(c)
Exclusión mutua	Lock()	pthread_mutex_t	binarysem b = 1
Concurrencia	Thread().start()	pthread_create	cobegin { }

En jBACI, mira **Globals**: `dentro` nunca pasa de 5, y la tabla de procesos muestra a los autos que esperan cupo.

Fuente: Pauta Examen Sistemas Operativos 2025-1, Problema 4 (15 pts.)

RESUMEN

jBACI en una diapositiva

Cuadro	Para qué sirve
Editor + Compile	Escribir el .cm y generar el PCode
Barra de botones	Go, Pause, Step Source y Step PCode
Tabla de procesos	Ver quién está activo, terminado o bloqueado
Ventana de proceso	Código con flecha verde y variables locales
Globals · Console	Valor del semáforo y salida del programa

Idea central: un semáforo deja pasar un proceso a la vez, y jBACI permite **verlo** paso a paso.